



第4章 非线性分类器

4.1 近邻法分类器

4.2 支持向量机

4.3 决策树

4.4 分类器的集成



4.3 决策树

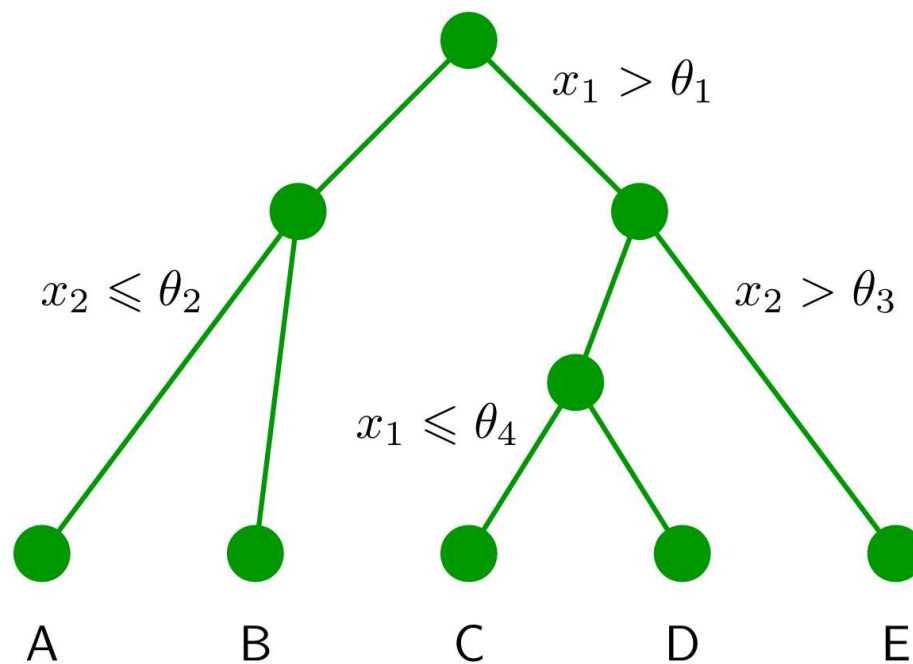
4.3.1 决策树概念

4.3.2 二叉树

4.3.3 决策树构建方法

4.3.4 过拟合与剪枝

4.3.5 问题讨论



一颗针对连续型变量的二叉决策树

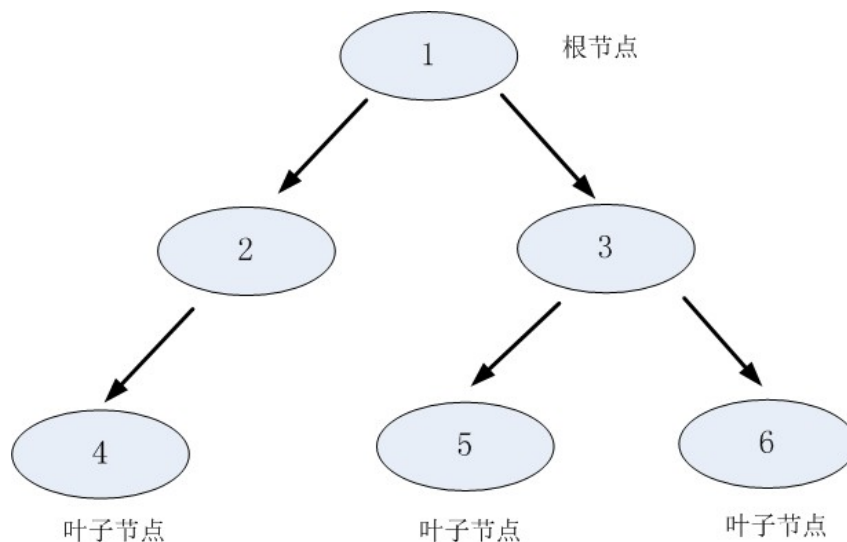


4.3 决策树

1、概念

➤ 树结构

- **树是由节点和边两种元素组成的结构，包括根节点、父节点、子节点和叶子节点等。**
- **父节点和子节点是相对的，子节点由父节点根据某一规则分裂而来，然后子节点作为新的父亲节点继续分裂，直至不能分裂为止。**
- **根节点**是没有父节点的节点，即初始分裂节点。
- **叶子节点**是没有子节点的节点。

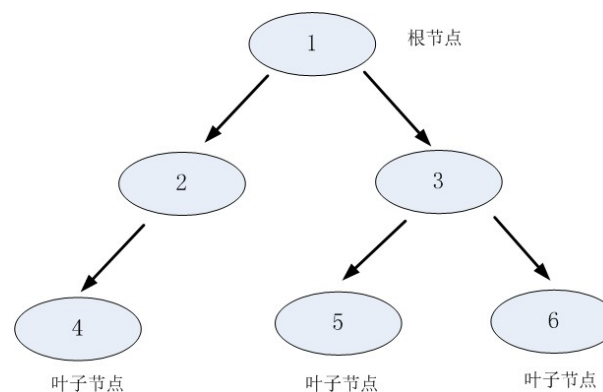




4.3 决策树

1、概念

- 决策树方法利用一定的训练样本，从数据中“学习”出决策规则，自动构造决策树，并利用决策树实现分类。
- 决策树利用**树结构**进行决策，每一个非叶子节点是一个判断条件，每一个叶子节点是结论。
- 从根节点开始，经过多次判断得出结论。

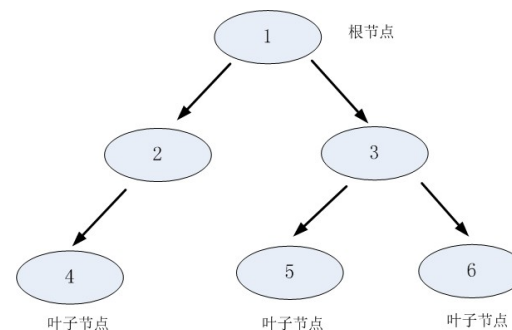




4.3 决策树

1、概念

➤ 对应树结构的相关概念



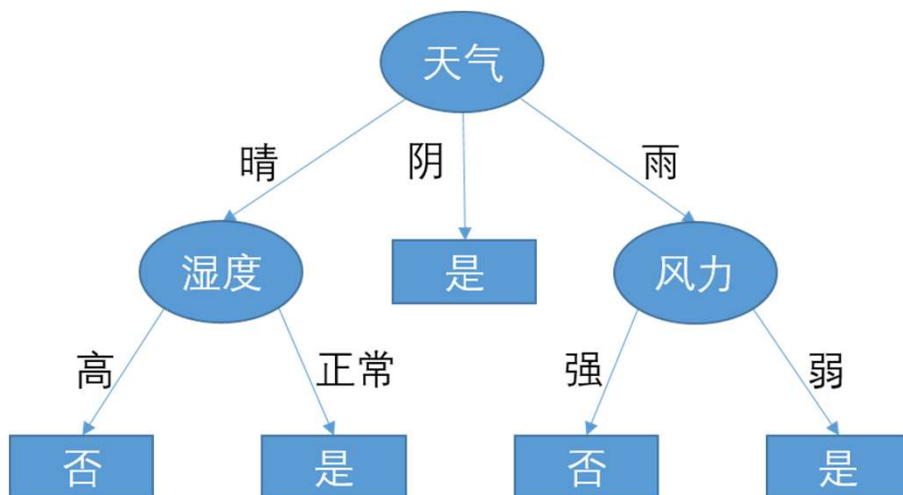
- ① **根节点(Root Node)**: 表示整个样本集合，且该节点可进一步划分成两个或多个子集。
- ② **拆分(Splitting)**: 表示将一个节点拆分成多个子集的过程。
- ③ **决策节点(Decision Node)**: 当一个子节点进一步被拆分成多个子节点时，该子节点就叫做决策节点。
- ④ **叶子节点(Leaf/Terminal Node)**: 无法再拆分的节点。
- ⑤ **剪枝(Pruning)**: 移除决策树中子节点的过程叫做剪枝，与拆分过程相反。
- ⑥ **分支/子树(Branch/Sub-Tree)**: 一棵决策树的一部分叫做分支或子树。
- ⑦ **父节点和子节点(Paren and Child Node)**: 一个节点被拆分成多个子节点，这个节点就叫做父节点；其拆分后的子节点也叫做子节点。



4.3 决策树

1、概念

下图绘出了一棵学习到的典型决策树。这棵决策树根据天气情况分类“星期六上午是否适合踢足球”。



可以将学习到的决策树表示为多个IF - THEN 规则。左图的决策树可以表示成以下的规则集：

- 规则1：如果 天气=晴 且 湿度=高，则 否；
- 规则2：如果 天气=晴 且 湿度=正常，则 是；
- 规则3：如果 天气=阴，则 是；
- 规则4：如果 天气=雨 且 风力=强，则 否；
- 规则5：如果 天气=雨 且 风力=弱，则 是。



4.3 决策树

2、二叉树

- **二叉树是最简单的决策树模型**
 - **每个非终止节点上采用线性分类器进行决策**
 - **每个非终止节点有两个分支，故称为二叉树**
 - **二叉树将多峰问题或多类问题转换为多级的线性问题**

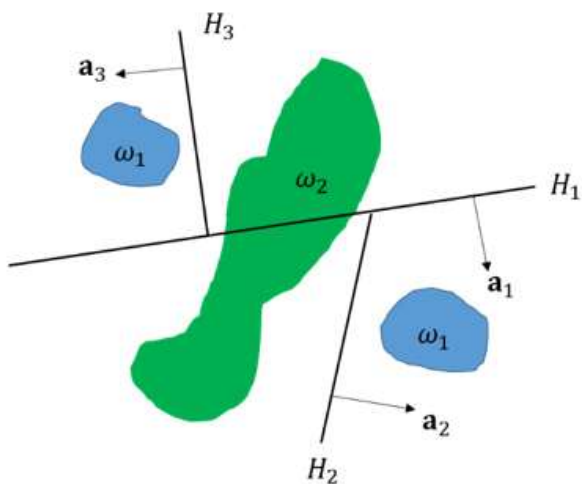


4.3 决策树

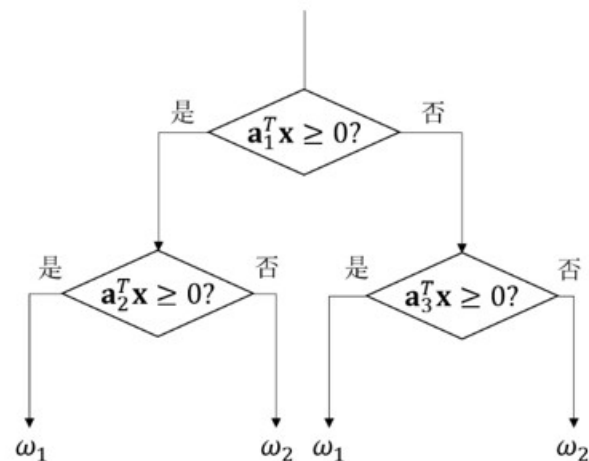
2、二叉树

➤ 二叉树算法步骤

- 采用线性判别函数 g ，决策面正负侧判别
- 设计线性分类器，将特征空间一分为二，成为两个子空间
- 对每个子空间，继续进行一分为二的操作，直至每个子空间中只含一类样本



二叉树在特征空间的表示



二叉树



4.3 决策树

2、二叉树

➤ 二叉树算法特点与问题

- 分段线性分类器
- 二叉树各节点（分段）可以优化设计
- 解决两类多峰问题
- 分类器设计相对复杂
- 第一个权向量影响后续所有权向量



4.3 决策树

3、决策树的构建方法

(1) 实例与问题分析

实例：右表为由若干客户（样本）组成的贷款申请训练数据，每一个客户都有四个属性（特征）：**年龄、是否工作、是否有自己的房子和信用情况**。现通过所给的训练数据学习一个贷款申请的决策树，当新的客户提出贷款申请时，根据申请人的属性利用决策树决定是否批准贷款申请。

尝试构建决策树

ID	年龄	是否工作	是否有自己的房子	信用情况	类别
1	青年	否	否	一般	否
2	青年	否	否	好	否
3	青年	是	否	好	是
4	青年	是	是	一般	是
5	青年	否	否	一般	否
6	中年	否	否	一般	否
7	中年	否	否	好	否
8	中年	是	是	好	是
9	中年	否	是	非常好	是
10	中年	否	是	非常好	是
11	老年	否	是	非常好	是
12	老年	否	是	好	是
13	老年	是	否	好	是
14	老年	是	否	非常好	是
15	老年	否	否	一般	否



4.3 决策树

3、决策树的构建方法

(1) 实例与问题分析

➤ 决策树构建一般步骤

- **特征（属性）选择：**从众多的特征中选择（属性）一个特征作为当前节点分裂的标准
- **决策树生成：**根据选择的特征评估标准，从上至下递归地生成子节点，直到数据集不可分则停止决策树生长
- **剪枝：**缩小树结构规模、缓解过拟合



4.3 决策树

3、决策树的构建方法

(1) 实例与问题分析



选择不同的特征可以产生不同的决策树。
如何确定特征选择的准则以更好地分类？

ID	年龄	是否工作	是否有自己的房子	信用情况	类别
1	青年	否	否	一般	否
2	青年	否	否	好	否
3	青年	是	否	好	是
4	青年	是	是	一般	是
5	青年	否	否	一般	否
6	中年	否	否	一般	否
7	中年	否	否	好	否
8	中年	是	是	好	是
9	中年	否	是	非常好	是
10	中年	否	是	非常好	是
11	老年	否	是	非常好	是
12	老年	否	是	好	是
13	老年	是	否	好	是
14	老年	是	否	非常好	是
15	老年	否	否	一般	否



4.3 决策树

3、决策树的构建方法

(1) 实例与问题分析



- **决策树学习的关键**在于如何选择最优划分属性（特征）
- **随着划分不断进行**，决策树的分支结点所包含的样本尽可能属于同一类别，即结点的“纯度”越来越高。
- **特征选择的目的（准则）**是使用某特征对数据集划分后，各数据子集的纯度要比划分前的纯度高，也即不确定性要比划分前数据集的不确定性低。

ID	年龄	是否工作	是否有自己的房子	信用情况	类别
1	青年	否	否	一般	否
2	青年	否	否	好	否
3	青年	是	否	好	是
4	青年	是	是	一般	是
5	青年	否	否	一般	否
6	中年	否	否	一般	否
7	中年	否	否	好	否
8	中年	是	是	好	是
9	中年	否	是	非常好	是
10	中年	否	是	非常好	是
11	老年	否	是	非常好	是
12	老年	否	是	好	是
13	老年	是	否	好	是
14	老年	是	否	非常好	是
15	老年	否	否	一般	否



4.3 决策树

3、决策树的构建方法

(2) ID3算法(Interactive Dichotomizer-3)

➤ 信息熵

- 在信息论中，**熵 (entropy)** 是随机变量不确定性的度量，熵越大，则随机变量的不确定性越大。**信息熵**是度量样本集合纯度最常用的一种指标。
- 设 X 是一个取有限个值的离散随机变量，其概率分布为： $P(X = x_i) = p_i, i = 1, 2, \dots, n$

则随机变量 X 的**熵定义为**：
$$H(X) = - \sum_{i=1}^n p_i \log p_i$$

上式中的对数一般以 2 为底，规定 $p_i = 0$ 时，定义 $0 \log 0 = 0$ 。

- 当熵中的概率由数据估计得到时，所对应的熵称为**经验熵(empirical entropy)**。



4.3 决策树

3、决策树的构建方法

(2) ID3算法(Interactive Dichotomizer-3)

➤ 经验熵的计算

定义贷款申请数据表中的数据为训练数据集 D ，其经验熵为 $H(D)$ ， $|D|$ 表示样本个数。设有 K 个类 $C_k, k = 1, \dots, K$ ， $|C_k|$ 为属于类 C_k 的样本个数，此时**经验熵公式**可以写为

$$H(D) = - \sum_k \frac{|C_k|}{|D|} \log \frac{|C_k|}{|D|}$$

根据此公式计算经验熵 $H(D)$ ，分析贷款申请样本表中的数据，最终放贷分类结果只有两类，即放贷和不放贷。根据表中的数据统计可知，在 15 个数据中，9 个数据的结果为放贷，6 个数据的结果为不放贷。**所以数据集 D 的经验熵 $H(D)$ 为：**

$$H(D) = -\frac{9}{15} \log \frac{9}{15} - \frac{6}{15} \log \frac{6}{15} = 0.971$$



4.3 决策树

3、决策树的构建方法

(2) ID3算法(Interactive Dichotomizer-3)

➤ 条件熵

条件熵 $H(Y | X)$ 表示在已知随机变量 X 的条件下随机变量 Y 的不确定性。随机变量 X 给定的条件下随机变量 Y 的条件熵(conditional entropy) 定义为 X 给定条件下 Y 的条件概率分布的熵对 X 的数学期望:

$$H(Y | X) = \sum_{i=1}^n p_i H(Y | X = x_i)$$

其中, $p_i = P(X = x_i)$, 当条件熵中的概率由数据估计 (特别是极大似然估计) 得到时, 为经验条件熵。



4.3 决策树

3、决策树的构建方法

(2) ID3算法(Interactive Dichotomizer-3)

➤ 信息增益

信息增益用来表示得知特征 X 的信息而使得类 Y 的信息不确定性减少的程度。

信息增益是相对于特征而言的。所以, A 特征对训练数据集 D 的信息增益 $\text{Gain}(D, A)$ 定义为 D 集合的经验熵 $H(D)$ 与特征 A 给定条件下 D 的经验条件熵 $H(D|A)$ 之差, 即:

$$\text{Gain}(D, A) = H(D) - H(D | A)$$

一般地, 熵 $H(D)$ 与条件熵 $H(D | A)$ 之差也称为互信息(mutual information)。信息增益等价于训练数据集中类与特征的互信息。



4.3 决策树

3、决策树的构建方法

(2) ID3算法(Interactive Dichotomizer-3)

➤ ID3 算法核心思路

- 选择当前使得信息增益最大的特征来划分数据集
- 选择该特性作为划分特征，则使用该特征划分得到的子集纯度更高，即不确定性更小
- ID3 算法自顶向下构造决策树，从“哪一个属性将在树的根节点被测试”开始，使用统计测试来确定每一个样本属性单独分类训练样例的能力。



4.3 决策树

3、决策树的构建方法

(2) ID3算法(Interactive Dichotomizer-3)

➤ ID3 算法决策树构建过程

- ID3 算法在决策树各个节点上，根据信息增益准则选择特征，递归地构建决策树。
- 从根节点(root node)开始，对节点计算所有可能的特征的信息增益，选择信息增益最大的特征作为节点的特征。
- 由该特征的不同取值建立子节点，再对子节点递归地调用以上方法，构建决策树；直到所有特征的信息增益均很小或没有特征可以选择为止，最后得到一个决策树。



4.3 决策树

3、决策树的构建方法

(2) ID3算法(Interactive Dichotomizer-3)

➤ ID3 算法步骤

输入：训练数据集 D ，特征集 A ，阈值 ε ；

输出：决策树 T 。

S1：若 D 中所有样本属于同一类，则 T 为单节点树，并将类作为该节点的类标记，输出 T ；

S2：若 $A = \emptyset$ ，则 T 为单节点树，并将 D 中样本数最大的类作为该节点的类标记，输出 T ；

S3：若 $A \neq \emptyset$ ，则计算 A 中各个特征对 D 的信息增益，选择信息增益最大的特征 A_{ig} ；

S4：若 A_{ig} 的信息增益小于阈值 ε ，则 T 为单节点树，并将 D 中样本数最大的类作为该节点的类标记，输出 T ；

S5：否则，对 A_{ig} 的每一种可能值 a_j ，按照 $A_{ig} = a_j$ 将 D 分割为若干非空子集 D_j ，将 D_j 中样本数最大的类作为标记，构建子节点，由节点及其子树构成树 T ，输出 T ；

S6：对第 j 个子节点，以 D_j 为训练集，以 $A - \{A_{ig}\}$ 为特征集合，递归调用 **S1~S5**，得到子树 T_j ，输出 T ；



4.3 决策树

3、决策树的构建方法

(2) ID3算法(Interactive Dichotomizer-3)

实例：右表为由若干客户（样本）组成的贷款申请训练数据，每一个客户都有四个属性（特征）：**年龄、是否工作、是否有自己的房子和信用情况**。现通过所给的训练数据学习一个贷款申请的决策树，当新的客户提出贷款申请时，根据申请人的属性利用决策树决定是否批准贷款申请。

尝试用ID3算法构建一颗决策树

ID	年龄	是否工作	是否有自己的房子	信用情况	类别
1	青年	否	否	一般	否
2	青年	否	否	好	否
3	青年	是	否	好	是
4	青年	是	是	一般	是
5	青年	否	否	一般	否
6	中年	否	否	一般	否
7	中年	否	否	好	否
8	中年	是	是	好	是
9	中年	否	是	非常好	是
10	中年	否	是	非常好	是
11	老年	否	是	非常好	是
12	老年	否	是	好	是
13	老年	是	否	好	是
14	老年	是	否	非常好	是
15	老年	否	否	一般	否



4.3 决策树

3、决策树的构建方法

(2) ID3算法(Interactive Dichotomizer-3)

➤ 用 ID3 算法构建贷款申请数据集的决策树

前有贷款申请数据集的经验熵的值 0.971，分别记年龄、是否工作、是否有自己的房子和信用情况为 A_1, A_2, A_3, A_4 ，现以 A_3, A_4 为例计算信息增益。
是否有自己的房子这个属性 (A_3) 对数据集的信息增益为：

$$H(Y | X) = \sum_{i=1}^n p_i H(Y | X = x_i)$$

$$\begin{aligned} \text{Gain}(D, A_3) &= H(D) - H(D|A_3) \\ &= 0.971 - \left[\frac{6}{15} H(D_1) + \frac{9}{15} H(D_2) \right] \\ &= 0.971 - \left[\frac{6}{15} \times 0 + \frac{9}{15} \times \left(-\frac{3}{9} \log \frac{3}{9} - \frac{6}{9} \log \frac{6}{9} \right) \right] \\ &= 0.971 - 0.551 = 0.420 \end{aligned}$$

其中 D_1, D_2 分别表示数据集中属性 A_3 取值分别为有自己的房子和没有自己的房子的样本子集。



4.3 决策树

3、决策树的构建方法

(2) ID3算法(Interactive Dichotomizer-3)

➤ 用 ID3 算法构建贷款申请数据集的决策树

信用情况这个属性 (A_4) 对数据集的信息增益为:

$$\begin{aligned}\text{Gain}(D, A_4) &= H(D) - H(D|A_4) \\ &= 0.971 - \left[\frac{4}{15} H(D_1) + \frac{6}{15} H(D_2) + \frac{5}{15} H(D_3) \right] \\ &= 0.971 - \left[\frac{4}{15} \times 0 + \frac{6}{15} \times \left(-\frac{4}{6} \log \frac{4}{6} - \frac{2}{6} \log \frac{2}{6} \right) + \frac{5}{15} \times \left(-\frac{1}{5} \log \frac{1}{5} - \frac{4}{5} \log \frac{4}{5} \right) \right] \\ &= 0.971 - 0.608 = 0.363\end{aligned}$$

其中 D_1, D_2, D_3 分别表示数据集中属性 A_4 取值为非常好、好和一般的样本子集。

同理可以计算出: $\text{Gain}(D, A_1) = H(D) - H(D|A_1) = 0.971 - 0.888 = 0.083$

$\text{Gain}(D, A_2) = H(D) - H(D|A_2) = 0.971 - 0.647 = 0.324$



4.3 决策树

3、决策树的构建方法

(2) ID3算法(Interactive Dichotomizer-3)

➤ 用 ID3 算法构建贷款申请数据集的决策树

从计算结果来看，是否有自己的房子 A_3 这个属性的信息增益最大，所以选择作为决策树的根节点特征。它将数据集划分为两个子集，有自己的房子的样本子集 D_1 和没有自己的房子的样本子集 D_2

D_1 只有同一类的样本点，所以它成为一个叶节点，节点的类标记为“是”。

对 D_2 则需要从特征中选择新的特征 A_1, A_2, A_4 ，计算各个特征的信息增益：

$$\text{Gain}(D_2, A_1) = H(D_2) - H(D_2|A_1) = 0.251$$

$$\text{Gain}(D_2, A_2) = H(D_2) - H(D_2|A_2) = 0.918$$

$$\text{Gain}(D_2, A_4) = H(D_2) - H(D_2|A_4) = 0.474$$



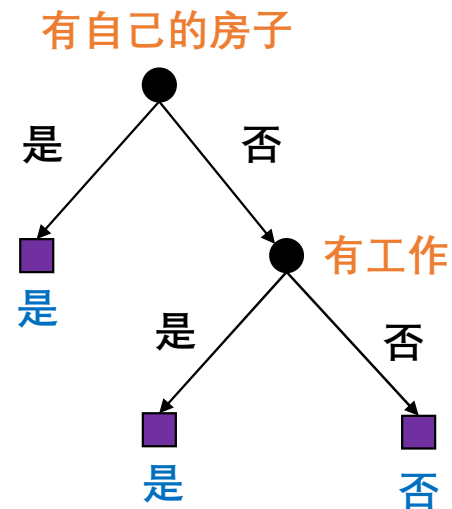
4.3 决策树

3、决策树的构建方法

(2) ID3算法(Interactive Dichotomizer-3)

➤ 用 ID3 算法构建贷款申请数据集的决策树

- 根据计算结果, A_2 信息增益最大, 将其作为节点的特征
- 其有两个取值, 引出两个子节点:
 - ① 对应“是”(有工作), 包含3个样本, 属同一类, 是叶节点, 类标记为“是”;
 - ② 对应“否”(无工作), 包含6个样本, 属同一类, 是叶节点, 类标记为“否”。
- 最终建立起的决策树用了两个特征



生成的决策树只用了两个特征



4.3 决策树

3、决策树的构建方法

(3) C4.5算法

➤ C4.5 算法的提出背景

信息增益的问题:

$$Gain(D, A) = H(D) - H(D|A)$$

- 信息增益会偏向取值较多的特征 $= H(D) - [p_1 \cdot H(D_1) + p_2 \cdot H(D_2) + \dots + p_n \cdot H(D_n)]$
- 信息增益值的大小本身是相对于训练数据集而言的，取值大小与数据集D是直接相关的，其值的大小也仅局限于该数据集D，并没有绝对意义
- 在分类问题困难时，也就是说在训练数据集经验熵大的时候，信息增益值会偏大，反之信息增益值会偏小



4.3 决策树

3、决策树的构建方法

(3) C4.5算法

➤ 信息增益比

特征 A 对训练数据集 D 的信息增益比定义为其信息增益与以 A 对训练数据集 D 进行划分的经验熵之比，即

$$\text{GainRatio}(D, A) = \frac{\text{Gain}(D, A)}{H_A(D)} \quad H_A(D) = - \sum_{i=1}^n \frac{|D_i|}{|D|} \log \frac{|D_i|}{|D|}$$

其中特征 A 有 n 种取值， $|D_i|$ 表示 A 的第 i 种取值对应的样本个数

C4.5 算法在生成决策树过程中采用信息增益比来选择特征，是 ID3 算法的改进



4.3 决策树

作业

实例：右表为由若干客户（样本）组成的贷款申请训练数据，每一个客户都有四个属性（特征）：**年龄、是否工作、是否有自己的房子和信用情况**。现通过所给的训练数据学习一个贷款申请的决策树，当新的客户提出贷款申请时，根据申请人的属性利用决策树决定是否批准贷款申请。

尝试用 C4.5 算法构建一颗决策树，写出计算过程

ID	年龄	是否工作	是否有自己的房子	信用情况	类别
1	青年	否	否	一般	否
2	青年	否	否	好	否
3	青年	是	否	好	是
4	青年	是	是	一般	是
5	青年	否	否	一般	否
6	中年	否	否	一般	否
7	中年	否	否	好	否
8	中年	是	是	好	是
9	中年	否	是	非常好	是
10	中年	否	是	非常好	是
11	老年	否	是	非常好	是
12	老年	否	是	好	是
13	老年	是	否	好	是
14	老年	是	否	非常好	是
15	老年	否	否	一般	否



4.3 决策树

3、决策树的构建方法

(4) CART算法

CART 算法使用基尼指数 (Gini index) 来选择最优特征, 同时决定该特征的最优二值划分点。假设分类问题中, 设有 K 个类, 样本属于第 k 类的概率为 p_k , 则概率分布的基尼指数定义为:

$$\text{Gini}(p) = \sum_{k=1}^K p_k(1 - p_k) = 1 - \sum_{k=1}^K p_k^2$$

对于二分类问题, 设属于第一类的概率为 p , 则 $\text{Gini}(p) = 2p(1 - p)$ 。

对于给定的样本集合 D , 假设有 K 个类 $C_k, k = 1, \dots, K$, $|C_k|$ 为属于类 C_k 的样本个数, $|D|$ 表示样本个数, 其基尼指数为:

$$\text{Gini}(D) = 1 - \sum_{k=1}^K \left(\frac{|C_k|}{|D|} \right)^2$$



4.3 决策树

3、决策树的构建方法

(4) CART算法

假设样本集合 D 根据特征 A 是否取某一可能值 a 被划分为 D_1 和 D_2 两部分，则在特征 A 的条件下，样本集合 D 的基尼指数定义为：

$$\text{Gini}(D, A) = \frac{|D_1|}{|D|} \text{Gini}(D_1) + \frac{|D_2|}{|D|} \text{Gini}(D_2)$$

基尼指数 $\text{Gini}(D)$ 表示集合 D 的不确定性，基尼指数 $\text{Gini}(D, A)$ 表示经 $A = a$ 分割后集合 D 的不确定性。基尼指数值越大，样本集合的不确定性也就越大，这一点跟熵相似。



4.3 决策树

3、决策树的构建方法

(4) CART算法

$$\text{Gini}(D, A) = \frac{|D_1|}{|D|} \text{Gini}(D_1) + \frac{|D_2|}{|D|} \text{Gini}(D_2)$$

➤ CART 算法步骤

输入：训练数据集 D ，阈值；

输出：CART 决策树 T 。

根据训练数据集 D ，从根节点开始，递归地对每个节点进行以下操作，构建二叉树：

S1：计算现有特征对该数据集 D 的基尼指数。即对每一个特征 A ，对其可能取的每个值 a ，根据样本是否满足 $A = a$ 将 D 划分 D_1 和 D_2 部分，计算 $A = a$ 时的基尼指数；

S2：在所有可能的特征 A 以及它们所有可能的划分点 a 中，选择基尼指数最小的特征及其对应可能的划分点作为最优特征和最优划分点。并从现节点生成两子节点，将训练数据集依特征分配到两子节点中去；

S3：对两个子节点递归地调用 **S1**、**S2**，直至满足条件；

S4：生成 CART 决策树。

算法停止条件是节点中样本个数小于预定阈值，或样本集的基尼指数小于预定阈值，或没有更多特征。



4.3 决策树

3、二叉树构建方法:

(4) CART算法

➤ 与ID3、C4.5算法区别

- ID3、C4.5: 属性 A 被选取建立决策树节点, 如果它有3个取值 A_1, A_2, A_3 , 会在决策树上建立一个三叉点, 决策树是多叉树。
- CART: 采用不停的二分。
 - a) 把属性 A 分成 $\{A_1\}$ 和 $\{A_2, A_3\}$ 、 $\{A_2\}$ 和 $\{A_1, A_3\}$ 、 $\{A_3\}$ 和 $\{A_1, A_2\}$ 三种情况, 找到基尼系数最小的组合, 比如 $\{A_2\}$ 和 $\{A_1, A_3\}$
 - b) 建立二叉树节点, 一个节点是 A_2 对应的样本, 另一个节点是 $\{A_1, A_3\}$ 对应的样本。
 - c) 上述操作没有把属性 A 的取值完全分开, 后面会对子节点继续选择属性 A 划分 A_1 和 A_3 。

与ID3、C4.5不同, 在ID3或C4.5的一棵子树中, 属性只会参与一次节点的建立。



4.3 决策树

3、二叉树构建方法:

(4) CART算法

➤ 对 CART 算法的讨论

- CART 生成的决策树是一棵二叉树，**本质是对特征空间进行二元划分**
- 能够对标量属性与连续属性进行分裂，可用于分类与回归
- **优点：**计算简单，易于理解，可解释性强，比较适合处理有缺失属性的样本，不仅能够处理不相关的特征，还能在相对短的时间内对大型数据源得出可行且效果良好的结果。
- **缺点：**不支持在线学习，在有新的样本产生后，决策树模型要重建；容易出现过拟合的现象，生成的决策树可能对训练数据有很好的分类能力，但对未知的测试数据却未必有很好的分类能力。



4.3 决策树

4、过拟合与剪枝

➤ 概念

- **过拟合 (over-fitting)**，也叫过学习 (over-learning)，指一个算法在训练数据上表现很好，但在测试数据上或未来的新数据上的表现很差
- **欠拟合**，指一个算法在训练数据上就没学好
- 一般来说，**欠拟合容易解决，过拟合不可避免**

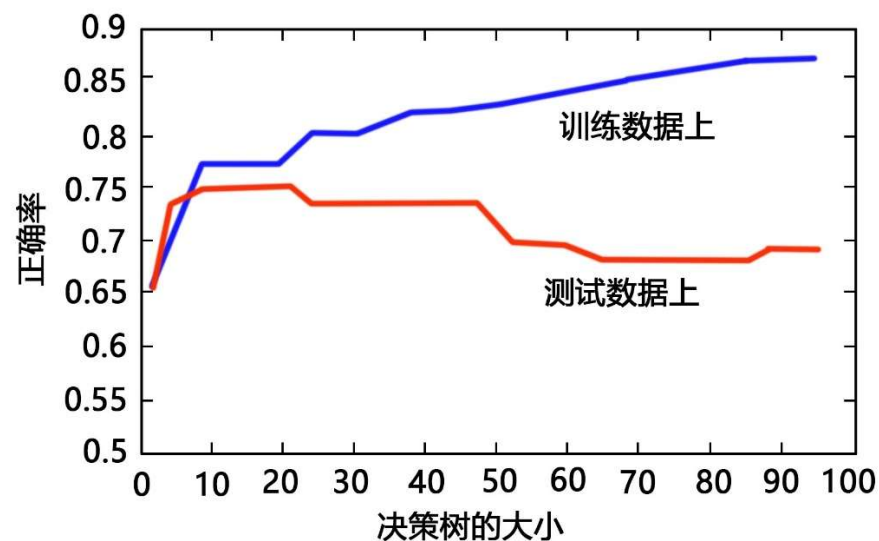


4.3 决策树

4、过拟合与剪枝

➤ 决策树的过拟合

- 在有限样本情况下，决策树生长得很大（树枝很多或很深），则可能会抓住有限样本中由于采样的偶然性或噪声带来的假象，**导致过拟合**
- 降低决策树过拟合风险的主要方法：控制决策树构建算法的终止条件、对决策树进行必要的剪枝
- 思考：**决策树欠拟合时怎么办？





4.3 决策树

4、过拟合与剪枝

➤ 剪枝 (pruning)

从已经生成的树上裁掉一些子树或叶节点，并将其根节点或父节点作为新的叶子节点，从而简化决策树模型，分为**先剪枝**和**后剪枝**

- **先剪枝**就是控制决策树的生长，在决策树构建过程中决定某节点是叶节点、还是继续分枝。
- **后剪枝**是在决策树得到充分生长之后，再对其进行修剪。从叶节点开始，如果合并具有相同父节点的叶节点后不会导致纯度明显降低，则执行合并。



4.3 决策树

4、过拟合与剪枝

➤ 剪枝 (pruning)

- 先剪枝

- ① **数据划分法**：将数据划分为训练样本和测试样本，基于训练样本对决策树进行生长，直至在测试样本上的分类错误率达到最小时停止生长。需要采用多次交叉验证。
- ② **阈值法**：预设一个信息增益阈值，小于阈值的节点停止生长。
- ③ **信息增益的统计显著性分析**：统计已有节点的信息增益的分布，继续生长后信息增益的统计分布变化不显著，则停止生长。



4.3 决策树

4、过拟合与剪枝

➤ 剪枝 (pruning)

- 后剪枝

- ① **减少分类错误率**：对比剪枝前后的分类错误率的改变，由此判断是否合并分支。
- ② **最小代价与复杂性折中**：折中考虑合并分枝后导致的错误率增加与复杂性减少，得到综合指标较优的决策树。可以定义代价函数进行优化。
- ③ **MDL 准则法 (Minimal Description Length)**：为构建最简单的树，对决策树编码，剪枝得到编码最短的树。



4.3 决策树

5、问题讨论

➤ 如何确定决策树生长的深度?

- 使用与训练样例截然不同的一套分离的样例，来评估通过后修剪方法从树上修剪节点的效用
- 使用所有可用数据进行训练，但进行统计测试来估计扩展（或修剪）一个特定的节点是否有可能改善在训练集合外的样本上的性能
- 使用一个明确的标准来衡量训练样例和决策树的复杂度，以便停止树增长



4.3 决策树

5、问题讨论

➤ 决策树如何处理连续值属性?

- 动态地定义新的离散值属性，即把连续值属性的值域分割为离散的区间集合
- 连续的属性分割成多个区间，而不是单一阈值的两个空间



4.3 决策树

5、问题讨论

➤ 决策树如何处理缺失值属性?

西瓜数据集 2.0 α

编号	色泽	根蒂	敲声	纹理	脐部	触感	好瓜
1	-	蜷缩	浊响	清晰	凹陷	硬滑	是
2	乌黑	蜷缩	沉闷	清晰	凹陷	-	是
3	乌黑	蜷缩	-	清晰	凹陷	硬滑	是
4	青绿	蜷缩	沉闷	清晰	凹陷	硬滑	是
5	-	蜷缩	浊响	清晰	凹陷	硬滑	是
6	青绿	稍蜷	浊响	清晰	-	软粘	是
7	乌黑	稍蜷	浊响	稍糊	稍凹	软粘	是
8	乌黑	稍蜷	浊响	-	稍凹	硬滑	是
9	乌黑	-	沉闷	稍糊	稍凹	硬滑	否
10	青绿	硬挺	清脆	-	平坦	软粘	否
11	浅白	硬挺	清脆	模糊	平坦	-	否
12	浅白	蜷缩	-	模糊	平坦	软粘	否
13	-	稍蜷	浊响	稍糊	凹陷	硬滑	否
14	浅白	稍蜷	沉闷	稍糊	凹陷	硬滑	否
15	乌黑	稍蜷	浊响	清晰	-	软粘	否
16	浅白	蜷缩	浊响	模糊	平坦	硬滑	否
17	青绿	-	沉闷	稍糊	稍凹	硬滑	否

假定 $(x, C(x))$ 是数据集 D 中的一个训练样例，并且其属性 A 的值 $A(x)$ 未知。

- 赋给它节点 n 的训练样例中该属性的最常见值
- 赋给它节点 n 的被分类为 $C(x)$ 的训练样例中该属性的最常见值
- 为 A 的每个可能值赋予一个概率，而不是简单地将最常见的值赋给 $A(x)$



第4章 非线性分类器

4.1 近邻法分类器

4.2 支持向量机

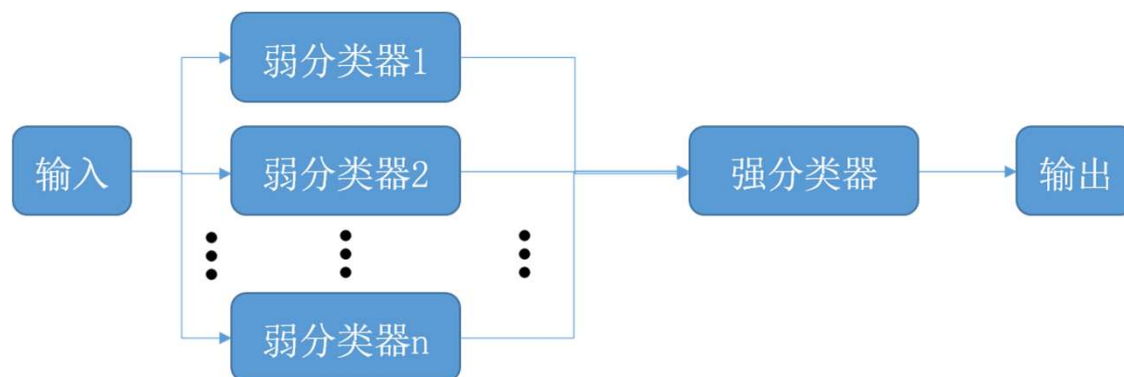
4.3 决策树

4.4 分类器的集成



4.4 分类器的集成

- 1、弱分类器的集成
- 2、随机森林
- 3、PAC 可学习理论与 AdaBoost
- 4、集成策略讨论





4.4 分类器的集成

1、弱分类器的集成

➤ 弱分类器概念

- 通常情况下，对于二分类问题，**弱分类器**是指识别错误率略小于 $1/2$ ，即准确率仅比随机猜测略高的分类器。
- 而**强分类器**是指识别准确率很高，并且能在多项式时间内完成的分类器。
- 问题：仅有一些弱分类器，能否获得强分类器的结果？

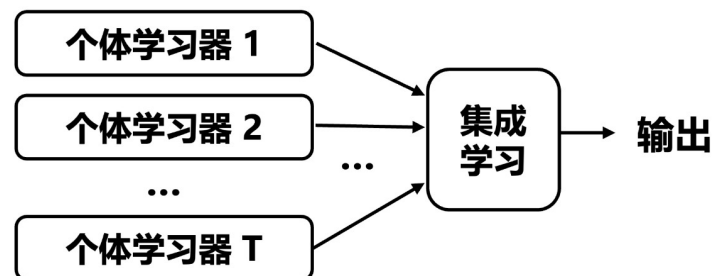


4.4 分类器的集成

1、弱分类器的集成

➤ 个体与集成

- 在有监督学习算法中，有时很难学习出一个稳定且在各个方面表现都较好的模型，而是只能得到多个有偏好的模型（仅在某些方面表现比较好，也叫弱模型）。
- 集成学习**就是组合多个弱模型以期得到一个更好更全面的强模型。
- 分类器集成，其实就是集成学习，通过构建并结合多个弱分类器来完成分类任务。一般结构是：先产生一组“个体学习器”，再用某种策略将它们结合起来。
- 集成学习潜在的思想：即便某一个弱分类器得到了错误的预测，其他的弱分类器也可以将错误纠正回来，从而提升分类性能。



三个臭皮匠顶个诸葛亮



4.4 分类器的集成

1、弱分类器的集成

➤ 举例分析

一个简单例子，在二分类问题中，假定 3 个分类器在三个样本中的表现如下图所示，其中 $\checkmark$ 表示分类正确， $\times$ 号表示分类错误，集成的结果通过投票产生。

测试例1 测试例2 测试例3				测试例1 测试例2 测试例3				测试例1 测试例2 测试例3			
h_1	✓	✓	×	h_1	✓	✓	×	h_1	✓	×	×
h_2	×	✓	✓	h_2	✓	✓	×	h_2	×	✓	×
h_3	✓	×	✓	h_3	✓	✓	×	h_3	×	×	✓
集群	✓	✓	✓	集群	✓	✓	×	集群	×	×	×
(a) 集群提升性能				(b) 集群不起作用				(c) 集群起负作用			

集成个体应：好而不同



4.4 分类器的集成

1、弱分类器的集成

➤ 错误率分析

考虑二分类问题 $y \in \{+1, -1\}$ 和真实分类器 $f(x)$ ，假设基分类器 $h_i(x)$ 的错误率 ϵ 为：

$$P(h_i(x) \neq f(x)) = \epsilon$$

假设集成通过简单投票法结合 T 个分类器，若有超过半数的基分类器正确，则分类就正确

$$H(x) = \text{sign}\left(\sum_{i=1}^T h_i(x)\right)$$

假设基分类器错误率相互独立，则集成分类器错误率：

$$P(H(x) \neq f(x)) = \sum_{k=0}^{\lfloor T/2 \rfloor} \binom{T}{k} (1 - \epsilon)^k \epsilon^{T-k} \leq \exp\left(-\frac{1}{2} T (1 - 2\epsilon)^2\right)$$



4.4 分类器的集成

1、弱分类器的集成

➤ 问题讨论

$$P(H(\mathbf{x}) \neq f(\mathbf{x})) = \sum_{k=0}^{\lfloor T/2 \rfloor} \binom{T}{k} (1 - \epsilon)^k \epsilon^{T-k} \leq \exp\left(-\frac{1}{2}T(1 - 2\epsilon)^2\right)$$

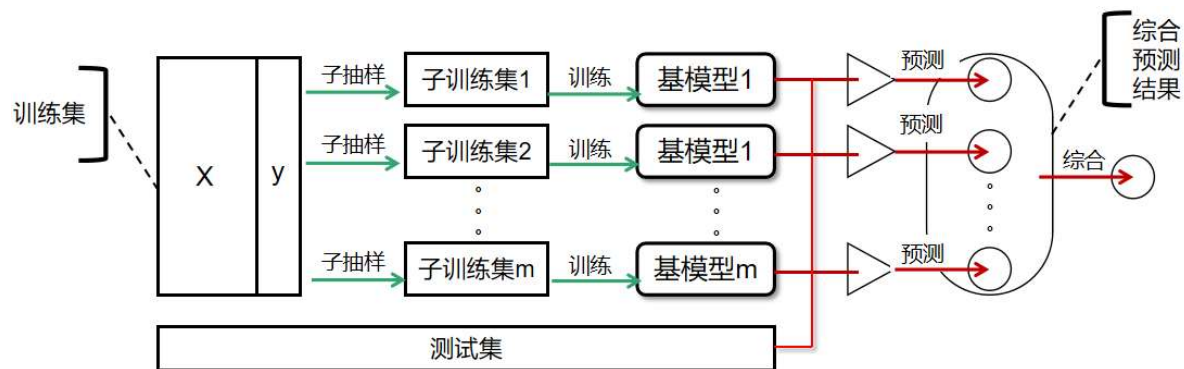
- 上面的分析有一个关键假设：基分类器的误差相互独立
- 现实任务中，个体分类器（基分类器）是为解决同一个问题训练出来的，显然不可能互相独立
- 事实上，个体分类器的“准确性”和“多样性”本身就存在冲突
- 如何产生“好而不同”的个体分类器是集成学习研究的核心
- 集成学习大致可分为两大类：**序列化方法**和**并行化方法**



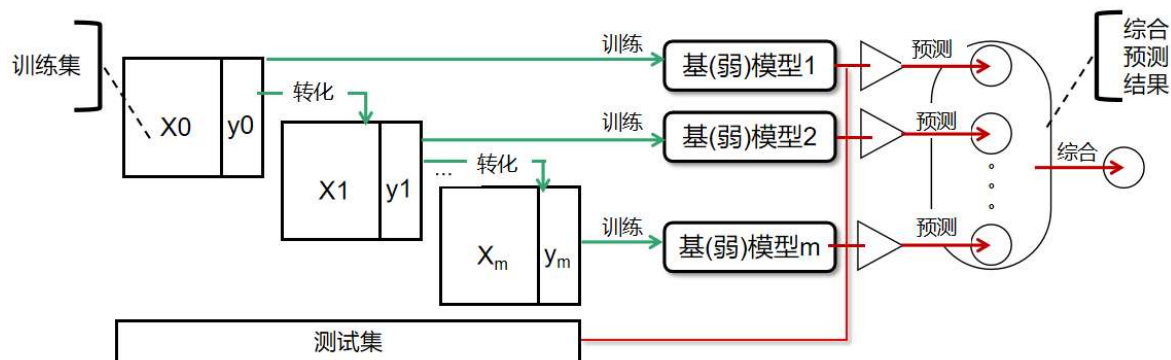
4.4 分类器的集成

1、弱分类器的集成

➤ 两类集成方法比较



Bagging
并行化方法



Boosting
序列化方法



4.4 分类器的集成

2、 随机森林 —— Bagging 法的典型应用

- Bagging (Bootstrap Aggregating) 在原始训练集上进行有放回的随机采样，得到若干个比较小的训练集并训练若干个模型，然后通过投票的方法进行模型集成。
- 通过不同模型的训练数据集的独立性来提高不同模型之间的独立性。



4.4 分类器的集成

2、随机森林

➤ 自助采样法 (bootstrap sampling)

- 给定包含 n 个样本的数据集
- 每次随机取出一个样本放入采样集，再把样本放回到初始数据集，这样下次采样时该样本仍有可能被选中
- 经过 n 次随机采样，即可得到含 n 个样本的采样集
- 初始训练集中有的样本在采样集中多次出现，有的则没有出现
- 可采样出 T 个含 n 个样本的采样集，基于每一个采样集训练出若干模型，再把这些模型集成，这就是 Bagging 的方法思路

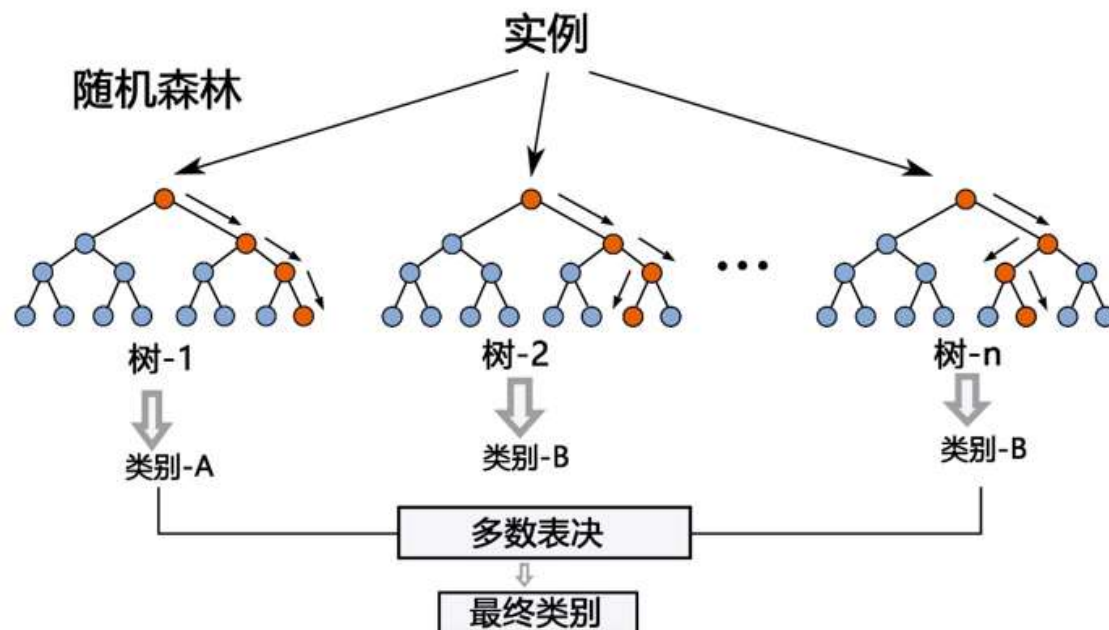
有
放
回
的
采
样



4.4 分类器的集成

2、随机森林

- 随机森林 (Random Forests) 是通过集成学习的方法将多棵决策树集成的一种算法。
- 每一棵决策树都是一个分类器，对于每一个输入样本，每棵决策树会有 1 个分类结果。
- 集成所有的分类投票结果，将投票次数最多的类别指定为最终的输出。





4.4 分类器的集成

2、随机森林

➤ 一般创建步骤

- ① 一个样本容量为 N 的样本集，每个样本有 M 个属性。有放回的抽取 N 次，每次抽取 1 个，形成 N 个样本。这 N 个样本用来训练一棵决策树。
- ② 在决策树的每个节点需要分裂时，随机从 M 个属性中选取 m 个属性，满足条件 $m \ll M$ 。然后从这 m 个属性中采用某种策略来选择 1 个属性作为该节点的分裂属性。
- ③ 决策树形成过程中每个节点都要按照步骤②来分裂，直到不能再分裂为止。注意整个决策树形成过程中没有进行剪枝。
- ④ 按照步骤①-③建立大量的决策树，构成随机森林。

随机抽样样本，
训练决策树

随机选取属性，
分裂节点

重复前一步，直
至不能再分裂

重复前三步，形
成森林



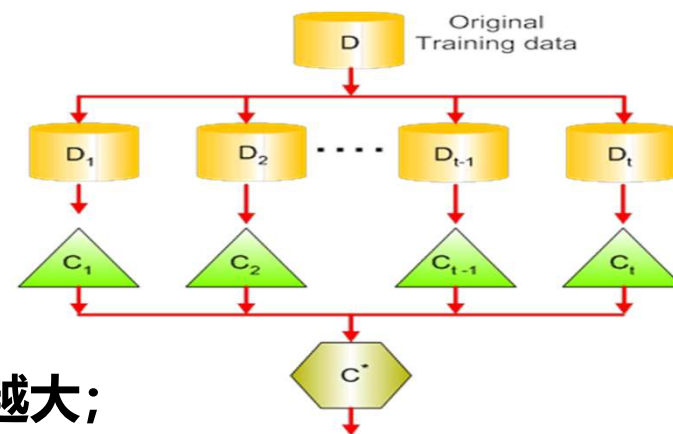
4.4 分类器的集成

2、随机森林

➤ 分类效果分析

影响随机森林分类效果的因素包括：

- ① 森林中任意两棵树的相关性：相关性越大，错误率越大；
- ② 森林中每棵树的分类能力：每棵树分类能力越强，整个森林的错误率越低；
- ③ 减少特征 m 的个数，树的相关性和分类能力都会下降；增大 m ，两者也会随之增大。因此，如何选择最优的 m ，也是随机森林需要确定的重要参数。





4.4 分类器的集成

2、随机森林

➤ 优点

- ① 准确率高;
- ② 能够运行在大数据集上;
- ③ 能够处理具有高维特征的样本输入, 不需要降维;
- ④ 能评估各个特征在分类问题上的重要性;
- ⑤ 对于缺省值问题, 也能够获得好的结果。

➤ 缺点

- ① 对分类问题表现好, 但对回归问题则表现一般, 因其无法给出精确连续的预测值。
- ② 包含大量随机产生的决策树, 构建过程难以控制, 只能通过尝试不同的参数来调整, 可解释性差。



4.4 分类器的集成

3、PAC 可学习理论与 AdaBoost

➤ 计算学习理论中的 PAC (Probably Approximately Correct) 学习

- 某个模型如果称为 PAC 可学习的 (learnable) , 则存在一种算法能够以很大的概率, 学到一个很高的精确度, 并且在多项式时间内完成。
- 由于需要对任意高的概率和精确度都可学习, 只有很少的模型是属于 PAC 可学习范畴的“强”可学习模型。“弱”可学习模型只能保证以任意高的概率学到比随机猜测好一点。在实践中, 获得一个弱学习算法容易, 获得一个强学习算法困难。
- Schapire 证明了“弱”可学习模型与“强”可学习模型两个概念等价。说明, 任何弱学习算法都可以被组合成一个强学习算法。



4.4 分类器的集成

3、PAC 可学习理论与 AdaBoost

➤ Boosting 方法概述

- Boosting 是能够把弱学习算法转化成强学习算法的一系列算法的集合。
- Boosting 方法是按照一定的顺序来训练不同的基模型，每个模型都针对前序模型的错误进行专门训练，从而增加不同基模型之间的差异性。
- Boosting 方法是一种非常强大的集成方法，只要基模型的准确率比随机猜测高，就可以通过集成方法来显著地提高集成模型的准确率。



4.4 分类器的集成

3、PAC 可学习理论与 AdaBoost

➤ Boosting 算法的工作机制

- 首先从初始训练集训练出一个弱分类器 1，根据弱分类器 1 的表现对训练样本分布进行调整，使得之前弱分类器 1 识别错误的训练样本在后面得到更多的重视。
- 然后基于调整后的训练集来训练弱分类器 2，如此重复进行，直到弱分类器数达到事先指定的数目 T ，
- 最终将这 T 个弱分类器通过集成策略进行整合，得到最终的强分类器。



4.4 分类器的集成

3、PAC 可学习理论与 AdaBoost

- Boosting 系列算法可以提高任意给定学习算法的准确度
- 训练过程为阶梯状，弱分类器按次序一一进行训练，弱分类器的训练集按照某种策略每次都进行一定的转化
- Boosting 中所有的弱分类器可以是不同类的分类器



4.4 分类器的集成

3、PAC 可学习理论与 AdaBoost

➤ Boosting 方法关键问题

1. 如何调整训练集用于训练弱分类器?
2. 如何将训练得到的各个弱分类器联合起来形成强分类器?



4.4 分类器的集成

3、PAC 可学习理论与 AdaBoost

AdaBoost 算法是 Boosting 方法的一种，针对以上两个关键问题，采用了以下策略：

- (1) **针对关键问题一：**使用加权后选取的训练数据代替随机选取的训练样本，这样将训练的焦点集中在比较难分的训练数据样本上；
- (2) **针对关键问题二：**将弱分类器联合起来，使用加权投票机制代替平均投票机制。让分类效果好的弱分类器具有较大的权重，而分类效果差的分类器具有较小的权重。



4.4 分类器的集成

3、PAC 可学习理论与 AdaBoost

具体来说，AdaBoost 利用一个基本分类器（看作弱分类器），根据它每次的输出结果，自适应地改变样本权重（已被正确分类的样本权值减小，被错分的样本权重提高），训练出一系列分类器，根据分类器的表现加权投票给出最终判决结果。

AdaBoost 算法是一种迭代式的训练算法，通过改变数据分布来提高弱分类器的差异。在每一轮训练中，增加分错样本的权重，减少对样本的权重，从而得到一个新的数据分布。



4.4 分类器的集成

3、PAC 可学习理论与 AdaBoost

➤ AdaBoost 算法原理

考虑一个两分类问题，数据点 x 是一个 n 维向量， $y \in \{-1, 1\}$ 代表类别。现在有 m 个训练样本，用 $(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)$ 来表示。在每一轮的迭代中，通过调用弱学习算法，产生一个新的弱分类器。

定义 $D_t(i)$ 代表第 t 轮迭代过程中，第 i 个样本点的权重。因为最初的 m 个训练样本是固定的，需要在每一轮的迭代过程中，调整每一个样本点的权重，来产生一个新的样本点分布，作为下一轮迭代的输入。



4.4 分类器的集成

3、PAC 可学习理论与 AdaBoost

➤ AdaBoost 算法原理

首先，对样本集中的所有样本点的权重进行初始化

$$D_1(i) = \frac{1}{m}, i = 1, 2, \dots, m$$

在第 $t = 1, 2, \dots, T$ 次的迭代中，第 i 个样本点的权重为 $D_t(i)$ 。调用弱学习算法，产生一个基分类器 h_t 。对于样本集中的每一个点，把基分类器 h_t 的分类结果与样本的实际类别 y 进行比较，计算该基分类器的错误率 ε_t 。对于第 i 个样本点，当 $h_t(x_i) \neq y_i$ 时，即表示该点的类别被判定错误。



4.4 分类器的集成

3、PAC 可学习理论与 AdaBoost

➤ AdaBoost 算法原理

- 定义每一轮迭代产生的基分类器应用于样本集分类的错误率为 ε_t ，可得

$$\varepsilon_t = \sum_{i=1}^m D_t(i) I(h_t(x_i) \neq y_i) / \sum_{i=1}^m D_t(i)$$

其中 $I(\cdot)$ 为指示函数，当括号里的表达式成立时为 1，否则为 0。

- 定义 α_t 为基分类器 h_t 的权重系数（用于后续加权投票集成）， α_t 可由下式计算

$$\alpha_t = \frac{1}{2} \ln\left(\frac{1 - \varepsilon_t}{\varepsilon_t}\right)$$

- 当 $\varepsilon_t = \frac{1}{2}$ 时， α_t 为 0。根据 Boosting 的思想，基分类器 h_t 的错误率是小于 50% 的，因此 α_t 是大于 0 的数。 ε_t 越小，意味着基分类器 h_t 的分类性能越好，权重 α_t 越大。



4.4 分类器的集成

3、PAC 可学习理论与 AdaBoost

➤ AdaBoost 算法原理

接着，对样本集中所有样本点的权重进行更新，产生一个新的分布 D_{t+1} 。对于分类正确的样本点，减小其权重；对于分类错误的样本点，增加其权重。这样在下一轮的迭代中，上一轮被分错的样本点将被重视。

D_{t+1} 可由下式计算:
$$D_{t+1}(i) = \begin{cases} D_t(i) \exp(-\alpha_t), & h_t(x_i) = y_i \\ D_t(i) \exp(\alpha_t), & h_t(x_i) \neq y_i \end{cases}$$

又因
$$y_i h_t(x_i) = \begin{cases} +1, & h_t(x_i) = y_i \\ -1, & h_t(x_i) \neq y_i \end{cases}$$

可将式化简为
$$D_{t+1}(i) = D_t(i) \exp[-\alpha_t y_i h_t(x_i)]$$



4.4 分类器的集成

3、PAC 可学习理论与 AdaBoost

➤ AdaBoost 算法原理

最后，在全部的 T 轮迭代完成之后，定义最终的强分类器为 H ，则把迭代过程中的所有基分类器 h_t 按照一定的规则组合起来

$$H(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(x) \right)$$

即对于一个样本点 x ，其分类结果就是 $H(x)$ 。



4.4 分类器的集成

3、PAC 可学习理论与 AdaBoost

➤ AdaBoost 算法的另一种解释

实际上, AdaBoost 算法也可以看作一种分步 (Stage-Wise) 优化的加性模型, 其损失函数定义为

$$\mathcal{L}(H) = \exp(-yH(x)) = \exp\left(-y \sum_{t=1}^T \alpha_t h_t(x)\right) \quad \leftarrow \begin{array}{l} y \in \{+1, -1\} \text{ 是样本 } x \text{ 对应的} \\ \text{真实标签, } H(x) \text{ 为组合后的} \\ \text{分类器} \end{array}$$

假设经过 $t-1$ 次迭代, 得到 $H_{t-1}(x) = \sum_{i=1}^{t-1} \alpha_i h_i(x)$

则第 t 次迭代的目标是找到 α_t 和 $h_t(x)$ 使得下面的损失函数最小(综合所有样本的损失函数)

$$\mathcal{L}(\alpha_t, h_t(x)) = \sum_{i=1}^m \exp\left(-y_i \underbrace{(H_{t-1}(x_i) + \alpha_t h_t(x_i))}_{H_t(x)}\right) \quad m \text{ 表示样本数量}$$



4.4 分类器的集成

3、PAC 可学习理论与 AdaBoost

➤ AdaBoost 算法的另一种解释

$$\mathcal{L}(\alpha_t, h_t(x)) = \sum_{i=1}^m \exp(-y_i(H_{t-1}(x_i) + \alpha_t h_t(x_i)))$$

令 $D_t(i) = \exp(-y_i H_{t-1}(x_i))$, 则损失函数可写为

$$\mathcal{L}(\alpha_t, h_t(x)) = \sum_{i=1}^m D_t(i) \exp(-y_i \alpha_t h_t(x_i))$$

$$= \sum_{y_i = h_t(x_i)} D_t(i) \exp(-\alpha_t) + \sum_{y_i \neq h_t(x_i)} D_t(i) \exp(\alpha_t)$$

$$y_i h_t(x_i) = \begin{cases} +1, & h_t(x_i) = y_i \\ -1, & h_t(x_i) \neq y_i \end{cases}$$

$$D_{t+1}(i) = D_t(i) \exp[-\alpha_t y_i h_t(x_i)]$$

$$\propto (1 - \varepsilon_t) \exp(-\alpha_t) + \varepsilon_t \exp(\alpha_t)$$

其中 ε_t 是基分类器 $h_t(x)$ 的错误率 $\varepsilon_t = \frac{\sum_{y_i \neq h_t(x_i)} D_t(i)}{\sum_{i=1}^m D_t(i)}$



4.4 分类器的集成

3、PAC 可学习理论与 AdaBoost

➤ AdaBoost 算法的另一种解释

$$\mathcal{L}(\alpha_t, h_t(x)) \propto (1 - \varepsilon_t) \exp(-\alpha_t) + \varepsilon_t \exp(\alpha_t)$$

求上式关于 α_t 的导数并令其为 0，可得到 $\alpha_t = \frac{1}{2} \ln\left(\frac{1 - \varepsilon_t}{\varepsilon_t}\right)$

可以证明，上述推导和 AdaBoost 算法原理中的定义是等价的。

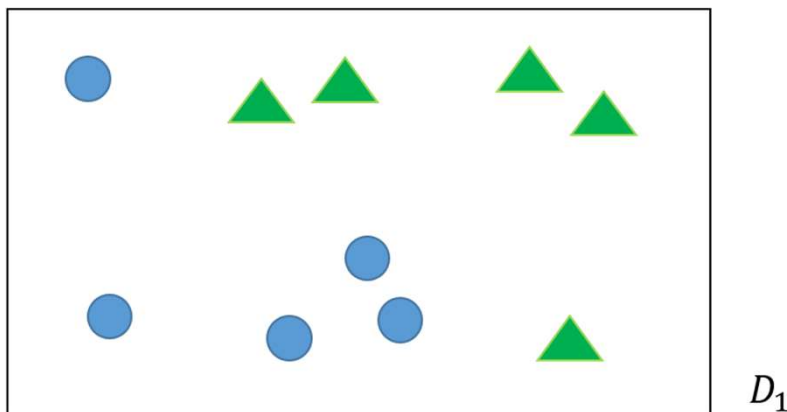


4.4 分类器的集成

3、PAC 可学习理论与 AdaBoost

➤ AdaBoost 算法的图例分析

用一个简单的例子，来看一下 Adaboost 的实现过程：



图中“▲”与“●”分别表示两种类别，用水平或垂直的直线作为分类器，来进行分类。



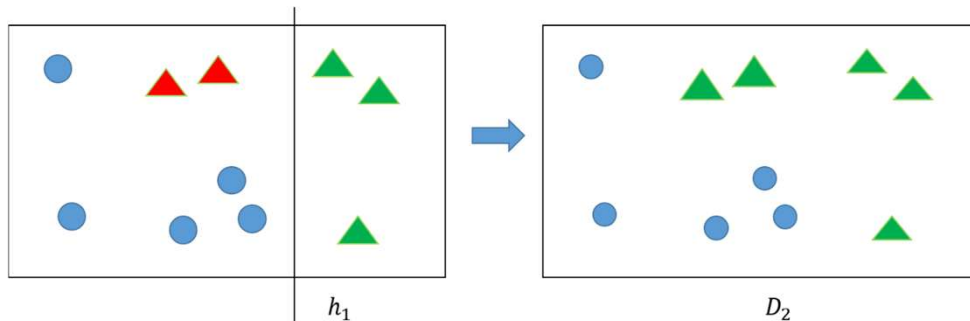
4.4 分类器的集成

3、PAC 可学习理论与 AdaBoost

➤ AdaBoost 算法的图例分析

第一步:

第一个基分类器



$$\varepsilon_t = \frac{\sum_{y_i \neq h_t(x_i)} D_t(i)}{\sum_{i=1}^m D_t(i)}$$

$$\alpha_t = \frac{1}{2} \ln\left(\frac{1 - \varepsilon_t}{\varepsilon_t}\right)$$

$$D_{t+1}(i) = D_t(i) \exp[-\alpha_t y_i h_t(x_i)]$$

初始时, 每一个样本的权重 $D_1(i) = 0.1, i = 1, 2, \dots, 10$, 图中 h_1 为一个基分类器。可

以看出有两个样本分类错误, 故 $\varepsilon_1 = 0.1 + 0.1 = 0.2, \alpha_1 = \frac{1}{2} \ln\left(\frac{1 - \varepsilon_1}{\varepsilon_1}\right) = 0.6931$

同时可以得到新的权重分布 D_2 。对于那两个分类错误的样本, 其权重变为 $D_2 = 0.1 \times e^{\alpha_1} = 0.2$; 对于其他分类正确的样本, 其权重变为 $D_2 = 0.1 \times e^{-\alpha_1} = 0.05$



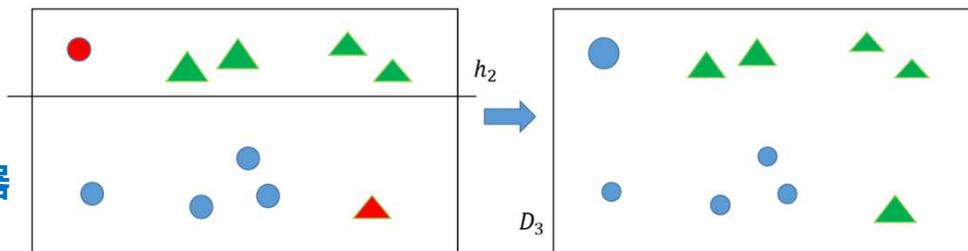
4.4 分类器的集成

3、PAC 可学习理论与 AdaBoost

➤ AdaBoost 算法的图例分析

第二步：

第二个基分类器



h_2 为基分类器，有两个样本分类错误，此时可以计算

$$\varepsilon_2 = \frac{0.0500 + 0.0500}{0.0500 \times 8 + 0.2000 \times 2} = 0.1250$$

$$\alpha_2 = \frac{1}{2} \ln \left(\frac{1 - \varepsilon_2}{\varepsilon_2} \right) = 0.9730$$

得到新的权值分布：

上轮分类错误的样本，在本轮中分类正确，权重变为： $D_3 = 0.2000 \times e^{-a_2} = 0.0756$

对于本轮中分类错误的样本，其权重更新为：

$$D_3 = 0.0500 \times e^{a_2} = 0.1323$$

对于其他样本，权重更新为：

$$D_3 = 0.0500 \times e^{-a_2} = 0.0189$$

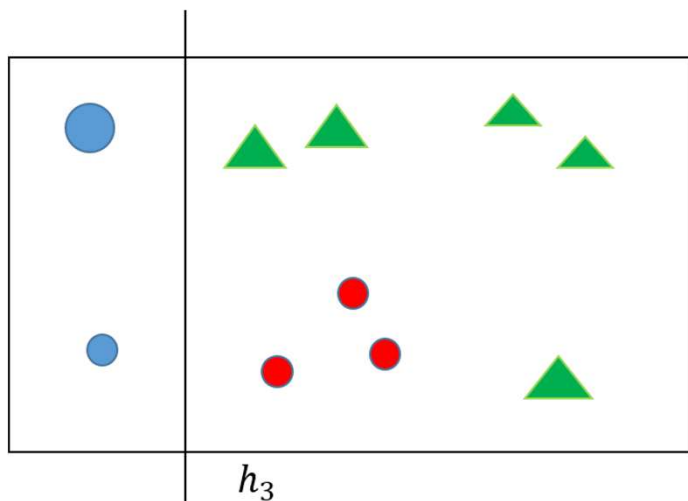


4.4 分类器的集成

3、PAC 可学习理论与 AdaBoost

➤ AdaBoost 算法的图例分析

第三步：



h_3 为新得到的基分类器，可以看出有三个样本分类错误，经计算可得：

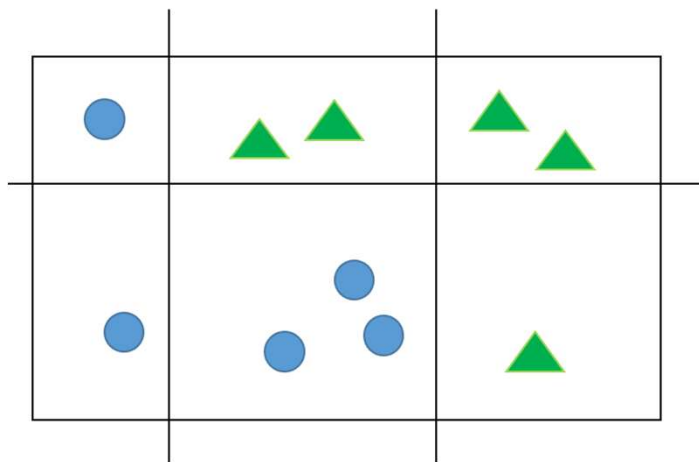
$$\varepsilon = 0.1071, \alpha_t = 1.0604$$



4.4 分类器的集成

3、PAC 可学习理论与 AdaBoost

➤ AdaBoost 算法的图例分析



整合所有的子分类器，可以得到最后的强分类器
即： $H(x)$

$$H(x) = \alpha_1 h_1(x) + \alpha_2 h_2(x) + \alpha_3 h_3(x)$$

$$H(x) = 0.6931h_1(x) + 0.9730h_2(x) + 1.0604h_3(x)$$

从结果中看，简单的弱分类器组合起来，也能得到比较好的分类效果。



4.4 分类器的集成

4、集成策略讨论

如何把多个弱分类器结果集成起来？ (Bagging 和 Boosting)

➤ 平均法

- 简单平均法
$$H(x) = \frac{1}{T} \sum_{i=1}^T h_i(x)$$
- 加权平均法
$$H(x) = \sum_{i=1}^T w_i h_i(x), w_i \geq 0, \sum_{i=1}^T w_i = 1$$

- 注：**
- ① 简单平均法是加权平均法的特例
 - ② 集成学习中的各种结果集成方法都可以看成是加权平均法的变种或特例
 - ③ 加权平均法可认为是集成学习研究的基本出发点
 - ④ 加权平均法未必一定优于简单平均法



4.4 分类器的集成

4、集成策略讨论

如何把多个弱分类器结果集成起来？（Bagging 和 Boosting）

➤ 投票法

- **绝对多数投票法**
$$H(x) = \begin{cases} c_j & \text{if } \sum_{i=1}^T h_i^j(x) > \frac{1}{2} \sum_{k=1}^l \sum_{i=1}^T h_i^k(x) \\ \text{rejection} & \text{otherwise} \end{cases}$$
- **相对多数投票法**
$$H(x) = c_{\arg \max_j \sum_{i=1}^T h_i^j(x)}$$
- **加权投票法**
$$H(x) = c_{\arg \max_j \sum_{i=1}^T w_i h_i^j(x)}$$



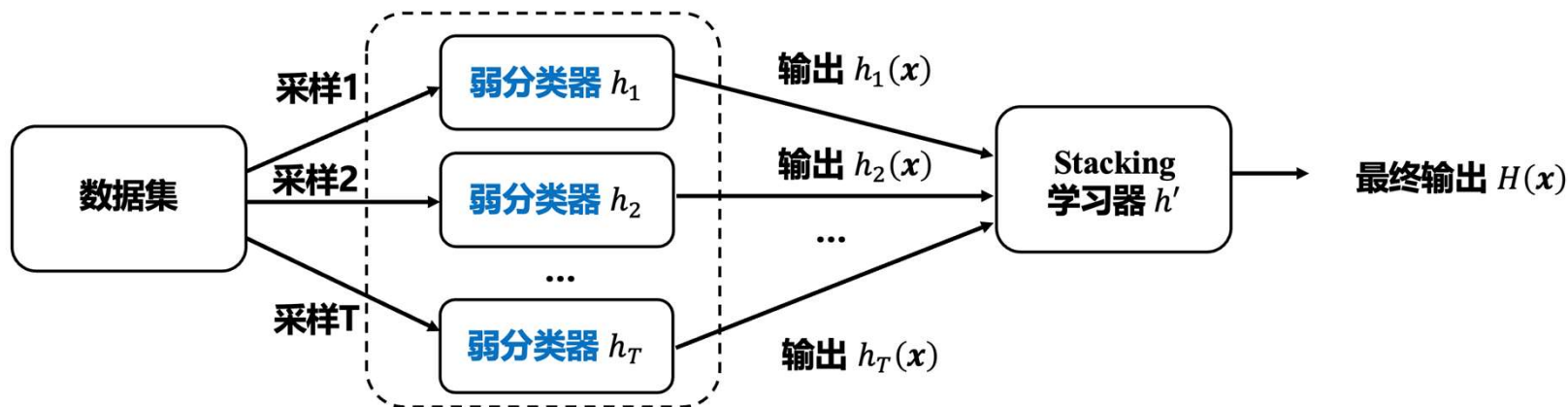
4.4 分类器的集成

4、集成策略讨论

如何把多个弱分类器结果集成起来？ (Bagging 和 Boosting)

➤ 学习法

- stacking $H(x) = h'(h_1(x), h_2(x), \dots, h_T(x))$





作业

➤ 4.1节近邻法分类器作业

1、已知：

甲类样本 4 个： $[2, 2]^T, [2, 3]^T, [1, 2]^T, [2, 1]^T$

乙类样本 4 个： $[-2, -2]^T, [-3, -2]^T, [-1, -2]^T, [-2, -3]^T$

试用最近邻分类器对未知样本 $[-1, -1]^T$ 和 $[3, 2]^T$ 进行分类，采用欧式距离度量。



作业

➤ 4.2节支持向量机作业

1、SVM 推导过程中 KKT 条件是什么，并对其进行分析。

2、已知：

输入样本 x	期望输出
$[0, 0]^T$	-1
$[0, 1]^T$	-1
$[0, 2]^T$	+1
$[2, 0]^T$	+1

且 $\alpha_1 = 0, \alpha_2 = 4, \alpha_3 = 3, \alpha_4 = 1$

请求解 w 和 b ，写出计算过程。



作业

➤ 4.3节决策树作业

将下面这组数据作为训练集，属性包括不浮出水面是否可以生存、是否有脚蹼，类别为是鱼类和非鱼类，利用ID3算法建立决策树

序号	不浮出水面是否可以生存	是否有脚蹼	类别
1	是	是	是
2	是	是	是
3	是	否	否
4	否	是	否
5	否	是	否



作业

➤ 4.4节集成学习作业

给定如下所示的训练数据，假设弱分类器是基于阈值的二分类器，其阈值使得该分类器在训练集上分类错误率最低。试用 AdaBoost 算法学习一个强分类器

序号	1	2	3	4	5	6	7	8	9	10
x	0	1	2	3	4	5	6	7	8	9
y	1	1	1	-1	-1	-1	1	1	1	-1



北京航空航天大学
BEIHANG UNIVERSITY

德才兼备 知行合一

The end